Activity – 01

Identify various Java IDEs and list the differences between Eclipse, NetBeans, and IntelliJ IDEA

Java IDEs (Integrated Development Environments) are software applications that provide comprehensive tools for writing, compiling, debugging, and managing Java programs. Instead of using separate tools for coding, compiling, and debugging, IDEs combine all these functionalities into a single interface, making development faster and more efficient. IDEs also provide features like syntax highlighting, code completion, error detection, version control integration, and project management tools.

Some commonly used Java IDEs include:

- Eclipse
- NetBeans
- IntelliJ IDEA
- BlueJ (for beginners)
- JDeveloper

Eclipse IDE

Eclipse is an open-source IDE widely used for Java development and other programming languages. It is highly flexible and supports plugins that allow developers to extend its functionality.

Key features:

- Open-source and free to use
- Highly customizable through plugins
- Strong support for large-scale enterprise applications
- Integrated debugging and testing tools
- Supports multiple languages (Java, C++, Python, etc.)

NetBeans IDE

NetBeans is another open-source IDE primarily used for Java development. It is known for its simplicity and ease of use, making it suitable for beginners as well as professionals.

Key features:

- Easy to install and use
- Built-in support for Java development (no extra plugins required)
- Strong GUI builder (Swing and JavaFX)

- Supports web and mobile development

IntelliJ IDEA

IntelliJ IDEA is a powerful and modern IDE developed by JetBrains. It is known for its intelligent code assistance and productivity features. It comes in two versions: Community (free) and Ultimate (paid).

Key features:

- Advanced code completion and suggestions
- Smart error detection and debugging
- Excellent support for frameworks (Spring, Hibernate)
- Built-in version control integration (Git, GitHub)
- High productivity with minimal configuration

Differences between Eclipse, NetBeans, and IntelliJ IDEA

Feature	**Eclipse**	**NetBeans**	**IntelliJ IDEA**
Type	Open-source	Open-source	Community (free) + Ultimate (paid)
Ease of Use	Moderate	Easy	Easy to Moderate
Performance	Can be slower with plugins	Stable	Fast and optimized
Plugin Support	Very strong	Moderate	Strong but structured
UI/UX	Less modern	Simple UI	Modern and user-friendly
Code Assistance	Basic	Moderate	Very advanced
Best For	Large enterprise apps	Beginners & students	Professional development

Conclusion

Java IDEs play a crucial role in modern software development by simplifying coding, debugging, and project management tasks. Eclipse, NetBeans, and IntelliJ IDEA each offer unique advantages depending on the user's needs. Eclipse is highly flexible and suitable for large projects, NetBeans is simple and beginner-friendly, while IntelliJ IDEA provides advanced features and is widely preferred in professional environments. Choosing the right IDE depends on project requirements, developer experience, and desired productivity level.

Activity – 02

Study Java coding standards and naming conventions

- Class names
- Method names
- Variable names

Class Names

Class names in Java follow **PascalCase (Upper Camel Case)**, where each word starts with a capital letter.

Key rules:

- Should be **nouns** (represent objects or entities)
- First letter of each word must be uppercase
- Should be meaningful and descriptive

Examples:

```
class StudentRecord {
}

class BankAccount {
}
```

Avoid:

```
class student { }  // not capitalized
class abc { }      // not meaningful
```

Method Names

Method names follow **camelCase**, starting with a lowercase letter.

Key rules:

- Should be **verbs** (represent actions)
- Use meaningful names describing functionality
- Avoid vague or short names

Examples:

```
void calculateTotal() {
}

int getMarks() {
```

```
    return marks;
}
```

Avoid:

```
void func() { }  // unclear
void m() { }      // too short
```

Variable Names

Variable names also follow **camelCase**.
Key rules:

- Should describe the data they store
- Start with lowercase
- Avoid single-letter names (except loops)
- Constants use uppercase with underscores

Examples:

```
int studentAge;
double totalAmount;
String firstName;

final double PI = 3.14;
```

Avoid:

```
int x;       // unclear
double a;    // poor naming
```

General Coding Practices

- Use proper indentation
- Keep code simple and readable
- Write comments where necessary
- Avoid long and complex methods
- Maintain consistent formatting

Why Coding Standards are Important

Coding standards are critical in software development for several reasons:
Importance:

- **Improves readability**
  Code becomes easy to understand even for new developers

- **Enhances maintainability**
  Well-structured code is easier to modify and update
- **Reduces errors**
  Clear naming reduces logical mistakes
- **Supports teamwork**
  Multiple developers can work on the same code without confusion
- **Saves time**
  Debugging and reviewing code becomes faster
- **Professional practice**
  Industry-level projects strictly follow coding standards

Example Comparison

Without standards:

```
class a {
   int x;
   void f() {
      System.out.println(x);
   }
}
```

With standards:

```
class Student {
   int studentAge;
   void displayAge() {
      System.out.println(studentAge);
   }
}
```

The second version is much clearer and easier to understand.

Conclusion

Java coding standards and naming conventions are essential for writing clean and professional code. By following proper rules for class names, method names, and variables, developers can create readable and maintainable programs. These standards are especially important in large projects and team environments, ensuring consistency and efficiency in software development.

Activity – 03

Compare and Contrast

- Method and constructor
- Default constructor and parameterized constructor.

In Java, methods and constructors are fundamental components used in class design. Although they may appear similar because both contain code blocks and can have parameters, their purpose and behavior are completely different. Understanding the difference between them is essential for writing structured and object-oriented programs.

Method and Constructor

A **method** is a block of code used to perform a specific task. It is called explicitly when needed and can return a value. A **constructor**, on the other hand, is a special type of method used to initialize objects. It is automatically called when an object of a class is created.

Key differences between method and constructor:

- A method must have a **return type** (void or any data type), whereas a constructor **does not have a return type**.
- A constructor has the **same name as the class**, while a method can have any valid name.
- Methods are **called explicitly**, but constructors are **invoked automatically** when an object is created.
- Methods can be called multiple times, whereas a constructor is called **only once per object creation**.

Example:

```
class Student {
   String name;

   // Constructor
   Student(String n) {
      name = n;
   }
   // Method
   void display() {
      System.out.println("Name: " + name);
   }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student s = new Student("Manish"); // constructor called
        s.display(); // method called
    }
}
```

In this example, the constructor initializes the object, while the method performs an action (displaying data).

Default Constructor and Parameterized Constructor

Constructors in Java can be classified into different types based on how they initialize objects.

A **default constructor** is a constructor with no parameters. If no constructor is defined in a class, Java automatically provides a default constructor. It initializes objects with default values.

A **parameterized constructor** is a constructor that accepts parameters. It allows initializing objects with specific values at the time of creation.

Key differences:

- Default constructor has **no parameters**, while parameterized constructor has **one or more parameters**.
- Default constructor assigns **default values**, whereas parameterized constructor assigns **user-defined values**.

Example:

```
class  Student  {
    String  name;
    int age;

    // Default constructor
    Student() {
        name = "Unknown";
        age = 0;
    }

    // Parameterized constructor
    Student(String n, int a) {
        name = n;
        age = a;
```

```
    }

    void display() {
        System.out.println(name + " " + age);
    }
}

public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // default constructor
        Student s2 = new Student("raja", 18); // parameterized constructor

        s1.display();
        s2.display();
    }
}
```

Output:

```
Unknown 0
raja
```

Use Cases

- Methods are used when we need to **perform operations** like calculations, printing data, or processing input.
- Constructors are used when we need to **initialize objects** at the time of creation.
- Default constructors are useful when **no initial values are provided**.
- Parameterized constructors are useful when **specific values are required during object creation**.

Conclusion

Methods and constructors serve different but equally important roles in Java. Methods define the behavior of objects, while constructors handle object initialization. Similarly, default constructors provide basic initialization, whereas parameterized constructors allow flexibility by assigning custom values. Understanding these differences helps in designing efficient and well-structured object-oriented programs.

Activity – 04

Study built-in Java packages

- java.lang
- java.util
- java.io

In Java, packages are collections of related classes and interfaces grouped together to organize code efficiently. Built-in packages (also called standard library packages) are provided by Java to perform common tasks such as handling strings, collections, input/output operations, and more. These packages reduce the need to write code from scratch and improve development speed.

java.lang

The java.lang package is the most fundamental package in Java. It is automatically imported into every Java program, so there is no need to explicitly import it.

Purpose:

- Provides basic classes required for Java programming
- Supports core functionalities like strings, mathematics, and system operations

Commonly used classes:

- **String** → used to handle text data
- **Math** → provides mathematical functions like sqrt(), pow()
- **System** → used for input/output operations (System.out.println)
- **Object** → parent class of all Java classes
- **Integer, Double** → wrapper classes for primitive data types

Example:

```
public class Demo {
   public static void main(String[] args) {
      String name = "Java";
      System.out.println(name.length()); // from String class
      System.out.println(Math.sqrt(16)); // from Math class
   }
}
```

Key point:

- No need to write import java.lang.*; because it is included by default.

java.util

The java.util package provides utility classes for data structures, collections, date handling, and random number generation. It is widely used in real-world applications.

Purpose:

- Provides tools for handling collections and data manipulation
- Supports data structures like lists, sets, and maps

Commonly used classes:

- **ArrayList** → dynamic array
- **HashMap** → key-value pair storage
- **HashSet** → collection of unique elements
- **Scanner** → used for input from user
- **Random** → generates random numbers

Example:

```
import java.util.*;

public class Demo {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Apple");
      list.add("Banana");
      System.out.println(list);
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter number: ");
      int n = sc.nextInt();
   }
}
```

Key point:

- Must be imported using import java.util.*;

java.io

The java.io package is used for input and output operations, especially for handling files and streams. It allows reading and writing data to files.

Purpose:

- Supports file handling and data streams
- Enables reading and writing data from/to files

Commonly used classes:

- **File** → represents file or directory
- **FileReader / FileWriter** → read/write character files

- **BufferedReader / BufferedWriter** → efficient reading/writing
- **PrintWriter** → prints formatted output

Example:

```
import java.io.*;

public class Demo {
    public static void main(String[] args) throws Exception {
        FileWriter fw = new FileWriter("test.txt");
        fw.write("Hello Java");
        fw.close();

        FileReader fr = new FileReader("test.txt");
        int ch;
        while ((ch = fr.read()) != -1) {
            System.out.print((char) ch);
        }
        fr.close();
    }
}
```

Key point:

- Used mainly for file handling and persistent data storage

Comparison of Packages

- java.lang → core package (auto imported, basic functions)
- java.util → utilities and data structures
- java.io → file handling and input/output operations

Advantages of Using Built-in Packages

- Saves development time
- Provides tested and reliable classes
- Improves code reusability
- Reduces complexity
- Makes programs more efficient

Conclusion

Built-in Java packages like java.lang, java.util, and java.io are essential for developing Java applications. They provide ready-made classes for core programming, data handling, and file operations. Understanding these packages helps developers write efficient, clean, and powerful programs without reinventing basic functionalities.

Activity – 05

Compare different types of loops in Java

- for loop
- while loop
- do-while loop

for loop

The **for loop** is used when the number of iterations is known in advance. It combines initialization, condition, and update in a single line, making it compact and easy to manage.

Syntax:

```
for (initialization; condition; update) {
   // code
}
```

Example:

```
public class Demo {
   public static void main(String[] args) {
      for (int i = 1; i <= 5; i++) {
         System.out.println(i);
      }
   }
}
```

Key points:

- Used when the number of repetitions is fixed
- Initialization, condition, and increment are written in one place
- Easy to read and maintain

Use cases:

- Printing numbers from 1 to 100
- Iterating through arrays or lists

while loop

The **while loop** is used when the number of iterations is not known in advance. It checks the condition before executing the loop body.

Syntax:

```
while (condition) {
   // code
}
```

Example:

```
public class Demo {
    public static void main(String[] args) {
        int i = 1;
        while (i <= 5) {
            System.out.println(i);
            i++;
        }
    }
}
```

Key points:

- Condition is checked before execution
- Loop may not execute at all if condition is false
- Suitable for dynamic conditions

Use cases:

- Reading input until user enters a specific value
- Processing data until a condition is met
- Infinite loops (while(true))

do-while loop

The **do-while loop** is similar to the while loop, but it executes the loop body at least once before checking the condition.

Syntax:

```
do {
    // code
} while (condition);
```

Example:

```
public class Demo {
    public static void main(String[] args) {
        int i = 1;
        do {
            System.out.println(i);
            i++;
        } while (i <= 5);
    }
}
```

Key points:

- Executes at least once even if condition is false

- Condition is checked after execution
- Useful when execution is required before validation

Use cases:

- Menu-driven programs
- Asking user input at least once
- Situations where first execution is mandatory

Comparison of Loops

- **for loop** → best when iterations are known
- **while loop** → best when condition is dynamic
- **do-while loop** → best when code must run at least once

Differences:

- for loop checks condition before execution and includes initialization/update in one line
- while loop checks condition before execution but requires manual initialization and update
- do-while loop checks condition after execution

Advantages of Using Loops

- Reduces code repetition
- Improves efficiency
- Makes programs shorter and cleaner
- Useful for handling large data

Conclusion

Loops are fundamental control structures in Java that allow repeated execution of code. The **for loop** is ideal for fixed iterations, the **while loop** is suitable for condition-based execution, and the **do-while loop** ensures at least one execution. Choosing the right loop improves code readability and efficiency in programming.

Activity – 06

Identify the advantages and disadvantages of the following OOP concepts

- Encapsulation
- Inheritance
- Abstraction
- Polymorphism

-Oriented Programming (OOP) is a programming paradigm that organizes software Object design around objects and classes. It improves code structure, reusability, and maintainability. The four core concepts of OOP are encapsulation, inheritance, abstraction, and polymorphism. Each concept has its own benefits and limitations depending on how it is used.

Encapsulation

Encapsulation is the process of wrapping data (variables) and methods (functions) into a single unit called a class. It also involves restricting direct access to data using access modifiers like private, protected, and public.

Advantages:

- Provides **data security** by restricting direct access
- Improves **code maintainability**

Disadvantages:

- Increases **code complexity** due to use of methods
- Slight performance overhead due to method calls

Example:

```
class Student {
    private int age;
    public void setAge(int a) { age
        = a;
    }
    public int getAge() {
        return age;
    }
}
```

Inheritance

Inheritance allows one class (child class) to acquire properties and methods of another class (parent class). It helps in code reuse and establishes a relationship between classes.

Advantages:

- Promotes **code reusability**
- Reduces code duplication

Disadvantages:

- Tight coupling between classes
- Changes in parent class may affect child classes

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}
```

Abstraction

Abstraction means hiding implementation details and showing only essential features to the user. It can be achieved using abstract classes and interfaces.

Advantages:

- Reduces complexity by hiding unnecessary details
- Improves code readability

Disadvantages:

- Difficult to design properly
- Can increase complexity for beginners

Example:

```
abstract class Shape {
    abstract void draw();
}
class Circle extends Shape { void
    draw() {
        System.out.println("Drawing  circle");
    }
}
```

Polymorphism

Polymorphism means “many forms.” It allows methods to behave differently based on the object or parameters. It is achieved through method overloading and method overriding.

Advantages:

- Improves code flexibility
- Supports dynamic method binding

Disadvantages:

- Can make code harder to understand
- Debugging becomes difficult

Example:

```
class Animal {
    void sound() {
        System.out.println("Animal  sound");
    }
}
class Cat extends Animal {
    void sound() {
        System.out.println("Cat  meows");
    }
}
```

Conclusion

OOP concepts such as encapsulation, inheritance, abstraction, and polymorphism are essential for designing efficient and scalable software systems. While they provide benefits like reusability, security, and flexibility, improper use can lead to increased complexity.

Activity – 07

Identify the advantages and disadvantages of the following OOP concepts

- Important methods of the Java Array Class.
- Important methods of the Java String Class.

In Java, the **Arrays class** and **String class** are part of the standard library and are widely used for handling data. The Arrays class provides utility methods to work with arrays, while the String class is used to manipulate text. Understanding their important methods helps in writing efficient and clean programs.

Important methods of the Java Arrays class

The Arrays class belongs to the java.util package and provides static methods to perform operations on arrays such as sorting, searching, and comparing.

Common methods:

- **sort()** → sorts the array in ascending order
- **binarySearch()** → searches an element in a sorted array
- **equals()** → compares two arrays
- **fill()** → fills array with a specific value
- **copyOf()** → copies an array to a new array
- **toString()** → converts array into string format

Example:

```
import java.util.Arrays;

public class Demo {
   public static void main(String[] args) {
      int[] arr = {5, 2, 8, 1};

      Arrays.sort(arr);
      System.out.println(Arrays.toString(arr));

      int index = Arrays.binarySearch(arr, 5);
      System.out.println("Index of 5: " + index);
   }
}
```

Output:
[1, 2, 5, 8]
Index of 5: 2
Key points:

- All methods are **static**, so no object is needed
- Mainly used for array manipulation

Important methods of the Java String class

The String class belongs to java.lang and is used to handle text. Strings in Java are immutable, meaning once created, they cannot be changed.

Common methods:

- **length()** → returns length of string
- **toUpperCase() / toLowerCase()** → changes case
- **charAt()** → returns character at specific index
- **substring()** → extracts part of string
- **equals()** → compares two strings
- **contains()** → checks if substring exists
- **replace()** → replaces characters
- **trim()** → removes spaces from start and end

Example:

```
public class Demo {
   public static void main(String[] args) {
      String str = "  Java Programming  ";

      System.out.println(str.length());
      System.out.println(str.trim());
      System.out.println(str.toUpperCase());
      System.out.println(str.substring(0, 4));
      System.out.println(str.contains("Java"));
   }
}
```

Output:

```
20
Java Programming
JAVAPROGRAMMING
Java
true
```

Key points:

- Strings are **immutable**
- Methods return new strings instead of modifying original

- Used in almost every Java program

Comparison

- Arrays class works with **arrays (fixed size data)**
- String class works with **text data**
- Arrays methods are static, String methods are instance methods

Use Cases

- Arrays methods → sorting marks, searching elements, copying data
- String methods → processing user input, formatting text, validation

Conclusion

The Java Arrays and String classes provide powerful built-in methods that simplify data handling and text manipulation. By using these methods, developers can write efficient and readable code without implementing complex logic manually. Understanding these methods is essential for solving real-world programming problems effectively.

Activity – 08

Identify and document how OOP design principles such as SRP, OCP, and ISP help in avoiding code smells

In software development, **code smells** are signs of poor design or bad coding practices that make code difficult to maintain, understand, or extend. Object-Oriented Design Principles help developers write clean, scalable, and maintainable code. Among these, **SRP (Single Responsibility Principle), OCP (Open/Closed Principle), and ISP (Interface Segregation Principle)** play a major role in preventing code smells.

Single Responsibility Principle (SRP)

SRP states that a class should have **only one reason to change**, meaning it should perform only one task or responsibility.

How it avoids code smells:

- Prevents **God Class** (a class doing too many things)
- Reduces complexity
- Makes code easier to test and maintain
- Improves readability

Example (Bad Design):

```
class Student {
   void calculateMarks() { }
   void printReport() { }
   void saveToDatabase() { }
}
```

Problem:

- One class handling multiple responsibilities → code smell

Improved Design (SRP applied):

```
class Student {
   void calculateMarks() { }
}

class Report {
   void printReport() { }
}

class Database {
   void save() { }
}
```

Now each class has a single responsibility.

Open/Closed Principle (OCP)

OCP states that software entities should be **open for extension but closed for modification**. This means we should be able to add new functionality without changing existing code.

How it avoids code smells:

- Prevents **frequent modification of existing code**
- Reduces risk of breaking working functionality
- Supports scalability and flexibility

Example (Bad Design):

```
class Shape {
    void draw(String type) {
        if (type.equals("circle")) {
            System.out.println("Circle");
        } else if (type.equals("square")) {
            System.out.println("Square");
        }
    }
}
```

Problem:

- Adding new shape requires modifying existing code

Improved Design (OCP applied):

```
abstract class Shape {
    abstract void draw();
}

class Circle extends Shape {
    void draw() {
        System.out.println("Circle");
    }
}

class Square extends Shape {
    void draw() {
        System.out.println("Square");
    }
}
```

Now new shapes can be added without modifying existing classes.

Interface Segregation Principle (ISP)

ISP states that a class should not be forced to implement interfaces it does not use. Instead of one large interface, use smaller, specific interfaces.

How it avoids code smells:

- Prevents **Fat Interface** problem
- Reduces unnecessary code implementation
- Improves flexibility and modularity

Example (Bad Design):

```
interface Machine {
   void print();
   void scan();
}
```

Problem:

- Not all machines need both methods

Improved Design (ISP applied):

```
interface Printer {
   void print();
}

interface Scanner {
   void scan();
}

class SimplePrinter implements Printer {
   public void print() {
      System.out.println("Printing");
   }
}
```

Now classes implement only what they need.

Summary of Code Smell Prevention

- SRP → avoids **large, complex classes**
- OCP → avoids **frequent code modification**
- ISP → avoids **unnecessary methods in interfaces**

Conclusion

OOP design principles like SRP, OCP, and ISP are essential for writing high-quality software. They help eliminate common code smells such as large classes, rigid code, and unnecessary dependencies. By applying these principles, developers can create systems that are flexible, maintainable, and easy to extend, which is critical in real-world software development.

Activity – 09

Compare and Contrast

- Static Binding and Dynamic Binding.
- Method overloading and Method overriding

In Java, binding refers to the process of linking a method call to its actual implementation. It plays an important role in determining how and when a method is executed. Binding can be classified into static binding and dynamic binding.

Static Binding and Dynamic Binding

Static binding (also called early binding) occurs at compile time. The method call is resolved by the compiler based on the reference type. It is used for methods that are not overridden.

Key points of static binding:

- Occurs at **compile time**
- Faster execution
- Does not support runtime polymorphism

Example:

```
class Demo {
   static void show() {
      System.out.println("Static method");
   }
}
public class Main {
   public static void main(String[] args) {
      Demo.show(); // resolved at compile time
   }
}
```

Dynamic binding (also called late binding) occurs at runtime. The method call is resolved based on the actual object type, not the reference type.

Key points of dynamic binding:

- Occurs at **runtime**
- Supports runtime polymorphism
- Used with overridden methods

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal sound");
   }
```

```
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
public class Main {
    public static void main(String[] args) {
        Animal obj = new Dog();
        obj.sound(); // resolved at runtime
    }
}
```

Difference summary:

- Static binding → compile time, faster, no polymorphism
- Dynamic binding → runtime, flexible, supports polymorphism

Method Overloading and Method Overriding

Both overloading and overriding are used to achieve polymorphism, but they work differently.

Method Overloading

Method overloading occurs when multiple methods have the same name but different parameters within the same class.

Key points:

- Same method name, different parameter list
- Occurs in the same class
- Compile-time polymorphism

Example:

```
class MathOperation {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Method Overriding

Method overriding occurs when a child class provides a specific implementation of a method already defined in the parent class.

Key points:

- Same method name and parameters
- Occurs between parent and child class
- Runtime polymorphism
- Requires inheritance

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal sound");
   }
}
class Cat extends Animal {
   void sound() {
      System.out.println("Cat meows");
   }
}
```

Difference summary:

- Overloading → same method name, different parameters, compile-time
- Overriding → same method signature, different implementation, runtime

Comparison Overview

- Static binding is related to **method overloading** (compile-time)
- Dynamic binding is related to **method overriding** (runtime)

Advantages

- Static binding → faster execution
- Dynamic binding → flexibility and extensibility
- Overloading → improves readability

Conclusion

Static and dynamic binding determine how method calls are resolved in Java, while method overloading and overriding define how methods behave. Static binding and overloading work at compile time, whereas dynamic binding and overriding work at runtime. Understanding these concepts is essential for implementing polymorphism and writing flexible, efficient Java programs.

Activity – 10

Study different types of packages in Java and explain the advantages of using packages in large programs

In Java, packages are used to group related classes and interfaces into a single unit. They help organize code in a structured way, making it easier to manage, maintain, and reuse. Packages also help avoid naming conflicts and improve code readability, especially in large applications.

Java provides two main types of packages: **built-in packages** and **user-defined packages**.

Types of Packages in Java

Built-in Packages

Built-in packages are pre-defined packages provided by Java. These packages contain ready-made classes and interfaces that developers can use directly.

Examples:

- java.lang → core classes like String, Math, System
- java.util → utility classes like ArrayList, Scanner
- java.io → file handling classes
- java.sql → database connectivity

Features:

- No need to create manually
- Saves development time
- Provides reliable and tested functionality

Example:

```
import java.util.ArrayList;

public class Demo {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("Java");
      System.out.println(list);
   }
}
```

User-defined Packages

User-defined packages are created by programmers to organize their own classes. These are useful in large projects where multiple classes need to be grouped logically.

Creating a package:

```
package mypackage;

public class Demo {
    public void show() {
        System.out.println("User-defined package");
    }
}
```

Using the package:

```
import mypackage.Demo;

public class Main {
    public static void main(String[] args) {
        Demo obj = new Demo();
        obj.show();
    }
}
```

Features:

- Helps organize large codebases
- Allows better modular design

Sub-packages

Java also allows creating packages inside other packages, known as sub-packages.
Example:
package com.company.project;

This structure helps in organizing large applications in a hierarchical manner.

Advantages of Using Packages in Large Programs

Using packages becomes very important when working on large-scale applications.

- **Code organization**
  Packages group related classes together, making code easy to navigate
- **Avoids name conflicts**
  Different classes can have the same name in different packages
- **Improves reusability**
  Classes in packages can be reused in other programs
- **Access control**
  Packages work with access modifiers (public, protected, default) to control visibility

- **Better maintenance**
  Easier to update and manage code in structured form
- **Supports modular programming**
  Large systems can be divided into smaller modules

Example Scenario

In a large project like an e-commerce application:

- com.app.user → handles user-related classes
- com.app.payment → handles payment processing
- com.app.order → manages orders

This structure makes the project clean and easy to manage.

Disadvantages

- Slight complexity in managing package structure
- Requires proper understanding of imports
- Incorrect package usage may cause errors

Conclusion

Packages in Java are essential for organizing and managing code efficiently, especially in large programs. Built-in packages provide ready-made functionality, while user-defined packages help developers structure their applications. By using packages properly, developers can create modular, reusable, and maintainable software systems.

Activity – 11

Differentiate error and exception in Java and identify commonly used system exceptions

In Java, both errors and exceptions are types of problems that occur during program execution. These are collectively known as **throwable objects**, handled using the Throwable class. However, errors and exceptions are different in nature, severity, and how they are handled.

An **error** represents serious problems that occur in the system and are generally beyond the control of the programmer. An **exception**, on the other hand, represents conditions that a program can handle and recover from.

Error in Java

Errors are critical issues that occur due to system-level problems. These are usually caused by the Java Virtual Machine (JVM) and are not meant to be handled by application code.

Key characteristics:

- Occur due to **system failures**
- Not recoverable
- Caused by JVM or hardware issues
- Rarely handled using try-catch

Examples of errors:

- **OutOfMemoryError** → occurs when JVM runs out of memory
- **StackOverflowError** → caused by excessive recursion
- **VirtualMachineError** → indicates JVM failure

Example:

```
public class Demo {
   public static void main(String[] args) {
      main(args); // infinite recursion → StackOverflowError
   }
}
```

Exception in Java

Exceptions are problems that occur during program execution but can be handled using proper code. They are caused by logical or runtime issues.

Key characteristics:

- Can be **handled using try-catch**
- Recoverable
- Caused by incorrect logic or invalid input

- Helps maintain program flow

Example:

```
public class Demo {
   public static void main(String[] args) {
      try {
         int a = 10 / 0;
      } catch (ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      }
   }
}
```

Types of Exceptions

Exceptions are mainly divided into two categories:

- **Checked Exceptions** → checked at compile time
  Example: IOException, SQLException
- **Unchecked Exceptions** → occur at runtime
  Example: ArithmeticException, NullPointerException

Difference between Error and Exception

- Error → system-level, not handled, critical
- Exception → application-level, handled, manageable

More clearly:

- Errors are **beyond programmer control**
- Exceptions are **within programmer control**

Commonly Used System Exceptions

Some commonly used exceptions in Java include:

- **ArithmeticException**
  Occurs when dividing a number by zero
- **NullPointerException**
  Occurs when accessing an object that is null
- **ArrayIndexOutOfBoundsException**
  Occurs when accessing an invalid index in an array
- **NumberFormatException**
  Occurs when converting invalid string to number
- **IOException**
  Occurs during input/output operations

Example:

```
public class Demo {
   public static void main(String[] args) {
      try {
         int[] arr = new int[3];
         arr[5] = 10; // invalid index
      } catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Invalid array index");
      }
   }
}
```

Advantages of Exception Handling

- Prevents program crash
- Maintains normal program flow
- Helps in debugging
- Improves program reliability

Disadvantages

- Improper handling can hide errors
- Adds extra code
- Can affect performance slightly

Conclusion

Errors and exceptions both indicate problems in a Java program, but they differ significantly. Errors are serious and usually not recoverable, while exceptions are manageable and can be handled using proper techniques. Understanding common system exceptions helps developers write safer and more reliable programs.

Activity – 12

Study Java wrapper classes and explain the concept of autoboxing and unboxing with examples

In Java, wrapper classes are used to convert primitive data types into objects. Java is an object-oriented language, but primitive types like int, char, double are not objects. Wrapper classes provide a way to use these primitive types as objects when required, especially in collections and APIs that work only with objects.

Each primitive data type has a corresponding wrapper class.
Primitive → Wrapper Class

- int → Integer
- char → Character
- double → Double
- float → Float
- long → Long
- short → Short
- byte → Byte
- boolean → Boolean

Wrapper classes are part of the java.lang package.

Need for Wrapper Classes

Wrapper classes are important because:

- Collections like ArrayList cannot store primitive types
- Provide utility methods for conversion and parsing
- Allow use of primitives as objects
- Support object-based operations

Example:

```
import java.util.*;

public class Demo {
   public static void main(String[] args) {
      ArrayList<Integer> list = new ArrayList<>();
      list.add(10); // primitive converted to object
      list.add(20);

      System.out.println(list);
```

```
    }
}
```

Without wrapper classes, this would not be possible.

Autoboxing

Autoboxing is the automatic conversion of a primitive data type into its corresponding wrapper object by the Java compiler.

Key points:

- Done automatically by compiler
- No need for manual conversion
- Improves readability

Example:

```
public class Demo {
    public static void main(String[] args) {
        int a = 10;

        Integer obj = a; // autoboxing
        System.out.println(obj);
    }
}
```

Here, the primitive int is automatically converted into an Integer object.

Unboxing

Unboxing is the reverse process of autoboxing. It converts a wrapper object back into a primitive data type automatically.

Key points:

- Also handled by compiler
- Converts object → primitive

Example:

```
public class Demo {
    public static void main(String[] args) {
        Integer obj = 20;

        int num = obj; // unboxing
        System.out.println(num);
    }
}
```

Here, the Integer object is converted back to a primitive int.

Manual Conversion (Before Autoboxing)

Before Java introduced autoboxing, conversion had to be done manually.
Example:

```
public class Demo {
    public static void main(String[] args) {
        int a = 10;

        Integer obj = Integer.valueOf(a); // manual boxing
        int num = obj.intValue();         // manual unboxing

        System.out.println(num);
    }
}
```

Now, this is handled automatically using autoboxing and unboxing.

Common Methods in Wrapper Classes

- parseInt() → converts String to int
- valueOf() → converts primitive to object
- toString() → converts value to String

Example:

```
public class Demo {
    public static void main(String[] args) {
        String str = "100";

        int num = Integer.parseInt(str);
        System.out.println(num);
    }
}
```

Conclusion

Wrapper classes play an important role in Java by allowing primitive data types to be treated as objects. Autoboxing and unboxing simplify the conversion process between primitives and objects, making code cleaner and easier to write. Understanding these concepts is essential for working with collections and modern Java applications.

Activity – 13

Identify popular Java frameworks used in industry (Spring, Hibernate, Struts, etc.) and list their basic features and applications

Java frameworks are pre-built libraries or platforms that provide a structured way to develop applications. They simplify development by offering reusable components, reducing boilerplate code, and improving productivity. In real-world software development, frameworks are heavily used to build scalable, secure, and maintainable applications.

Some of the most popular Java frameworks used in industry are **Spring, Hibernate, and Struts**.

Spring Framework

Spring is one of the most widely used Java frameworks. It is mainly used for building enterprise-level applications and supports features like dependency injection and aspect-oriented programming.

Basic features:

- **Dependency Injection (DI)** → reduces tight coupling between classes
- **Inversion of Control (IoC)** → manages object creation automatically
- **Spring Boot** → simplifies application setup and configuration
- **Built-in support for web applications (Spring MVC)**
- Integration with databases and other frameworks

Example (Spring Boot main class):

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class App {
   public static void main(String[] args) {
      SpringApplication.run(App.class, args);
   }
}
```

Applications:

- Web applications
- REST APIs
- Enterprise systems
- Microservices architecture

Hibernate Framework

Hibernate is an Object-Relational Mapping (ORM) framework used to simplify database operations. It maps Java objects to database tables.

Basic features:

- **ORM (Object-Relational Mapping)**
- Eliminates need for writing SQL queries manually
- Supports database independence
- Automatic table creation and data mapping
- Provides caching for better performance

Example:

```
@Entity
class Student {
   @Id
   int id;
   String name;
}
```

Applications:

- Database-driven applications
- Enterprise applications
- Data management systems

Struts Framework

Struts is a web application framework based on the Model-View-Controller (MVC) architecture. It is mainly used for developing traditional web applications.

Basic features:

- Follows **MVC architecture**
- Separates business logic from presentation
- Uses configuration files (XML-based)
- Supports form validation
- Provides reusable components

Applications:

- Web applications
- Enterprise-level systems
- Legacy Java projects

Comparison of Frameworks

- Spring → most modern and widely used, supports full-stack development
- Hibernate → focuses on database interaction
- Struts → older framework, mainly for web applications

Advantages of Using Frameworks

- Reduces development time
- Provides ready-made components
- Improves code quality and structure
- Supports scalability and maintainability
- Widely used in industry

Disadvantages

- Learning curve for beginners
- Adds dependency on framework
- Can increase application size

Conclusion

Java frameworks like Spring, Hibernate, and Struts play a major role in modern software development. Spring is widely used for building scalable applications, Hibernate simplifies database operations, and Struts provides structured web development. Using frameworks helps developers build efficient, maintainable, and professional applications suitable for industry standards.